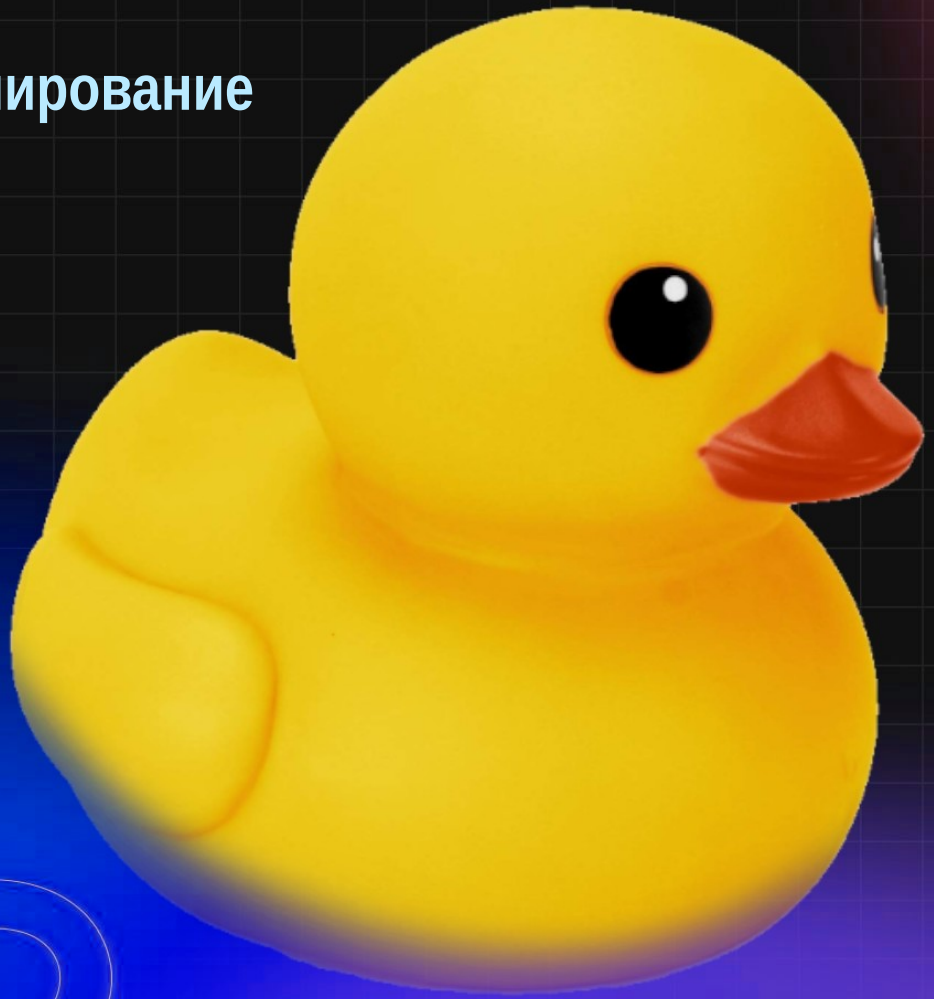
Программирование

2 семестр

2026



Ввод-вывод (NIO)

IO / NIO

ИТМО

- Преимущества NIO
  - больше производительность
  - поддержка буферов с прямым доступом
  - поддержка неблокируемых каналов
  - поддержка мультиплексируемых каналов (selectors)
  - поддержка асинхронных каналов

Java

Блокируемый / неблокируемый I/O

ИТМО

- Блокируемый
  - Операция ввода-вывода блокирует выполнение
  - Нельзя продолжить, пока она не закончится
- Неблокируемый
  - Операция ввода-вывода возвращает 0, если нет данных
  - Можно сделать что-то еще, и попробовать еще раз

Java

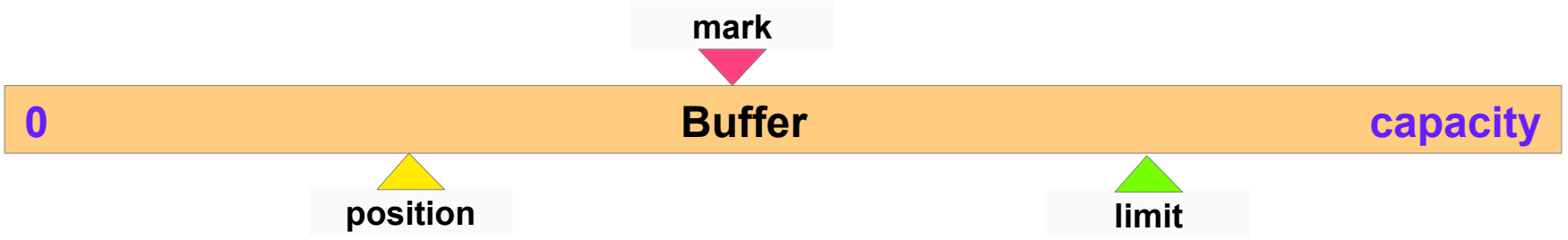
Синхронный / асинхронный I/O

ИТМО

- Синхронный
  - Операция ввода-вывода вернет данные в том же потоке
  - Последовательное выполнение: запрос - ответ - обработка
- Асинхронный
  - Операция ввода-вывода:
    - возвращает контейнер для будущего результата (Future)
    - получает объект-обработчик результата (CompletionHandler)
  - Не нужно ждать для отправки нового запроса

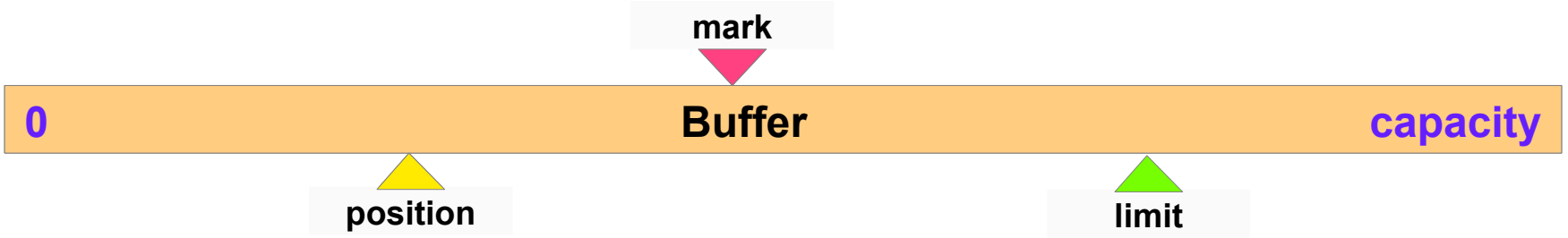
Java

java.nio - Buffer



- java.nio.Buffer — контейнер для хранения данных
  - capacity - емкость (максимальный размер)
  - limit - сколько можно записать (не больше емкости) или прочитать (не больше, чем записано)
  - position - текущая позиция
  - mark - метка

java.nio - Buffer



- Создание буфера:
  - allocate(capacity)
  - allocateDirect(capacity)
  - wrap(array[])

- Методы:
  - limit(lim) и position(pos)
  - mark() и reset() mark <-> position
  - clear() - limit = capacity, position = 0
  - compact - все недочитанное - в начало буфера
  - flip() - limit = position, position = 0
  - rewind() - position = 0

Запись и чтение данных

ИТМО

- методы get и put
  - get - чтение из буфера
  - put - запись в буфер

- Абсолютная индексация (явное указание индекса)
  - позиция не меняется
  - только одиночные операции

- Относительная индексация (по текущей позиции)
  - позиция смещается после операции
  - одиночные и групповые

Java

Запись и чтение данных

- **запись**

  clear();

  while () { put(byte); }

- чтение

  flip();

  while(hasRemaining()) { get(); }

Запись и чтение данных

- **запись**

  clear();

  while () { put(byte); }

- чтение

  flip();

  while(hasRemaining()) { get(); }

Запись и чтение данных

ИТМО

- **запись**

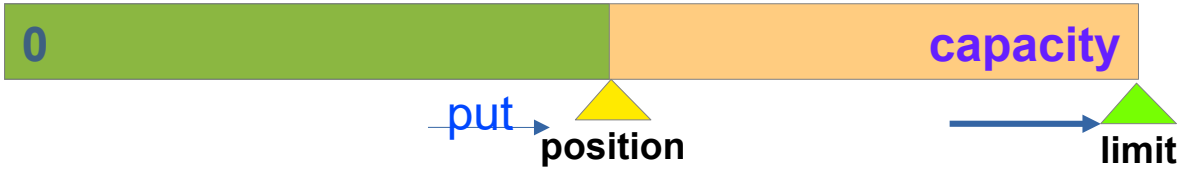
  clear();

  while () { **put(byte);** }

- чтение

  flip();

  while(hasRemaining()) { get(); }



Java

Запись и чтение данных

ИТМО

- **запись**

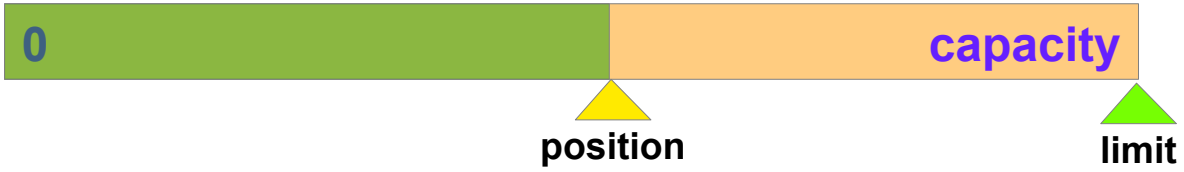
  clear();

  while () { put(byte); }

- чтение

  flip();

  while(hasRemaining()) { get(); }



Java

Запись и чтение данных

- **запись**

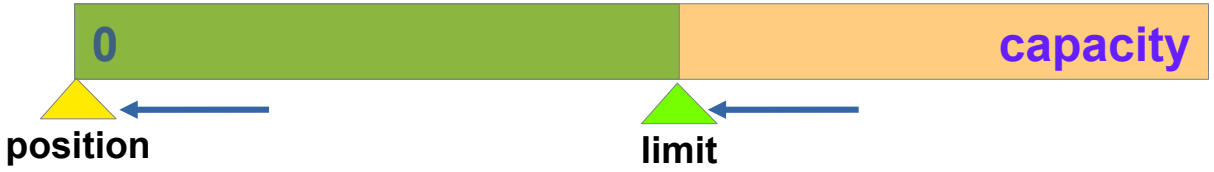
  clear();

  while () { **put(byte);** }

- чтение

  flip();

  while(hasRemaining()) { get(); }



Запись и чтение данных

- запись

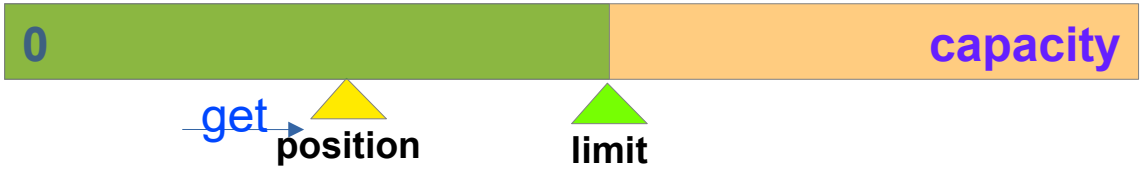
  clear();

  while () { put(byte); }

- **чтение**

  flip();

  while(hasRemaining()) { **get();** }



Классы-потомки Buffer

- ByteBuffer
- CharBuffer
- IntBuffer, ShortBuffer, LongBuffer, FloatBuffer, DoubleBuffer
  - getInt(), putInt(), getShort(), putShort() ...
- java.nio.ByteOrder
  - ByteOrder.BIG_ENDIAN
  - ByteOrder.LITTLE_ENDIAN
  - nativeOrder()
  - order()

java.nio.charset

ИТМО

- Класс Charset - кодировка символов
  - методы
  - CharBuffer decode(ByteBuffer b)
  - ByteBuffer encode(CharBuffer c)

Java

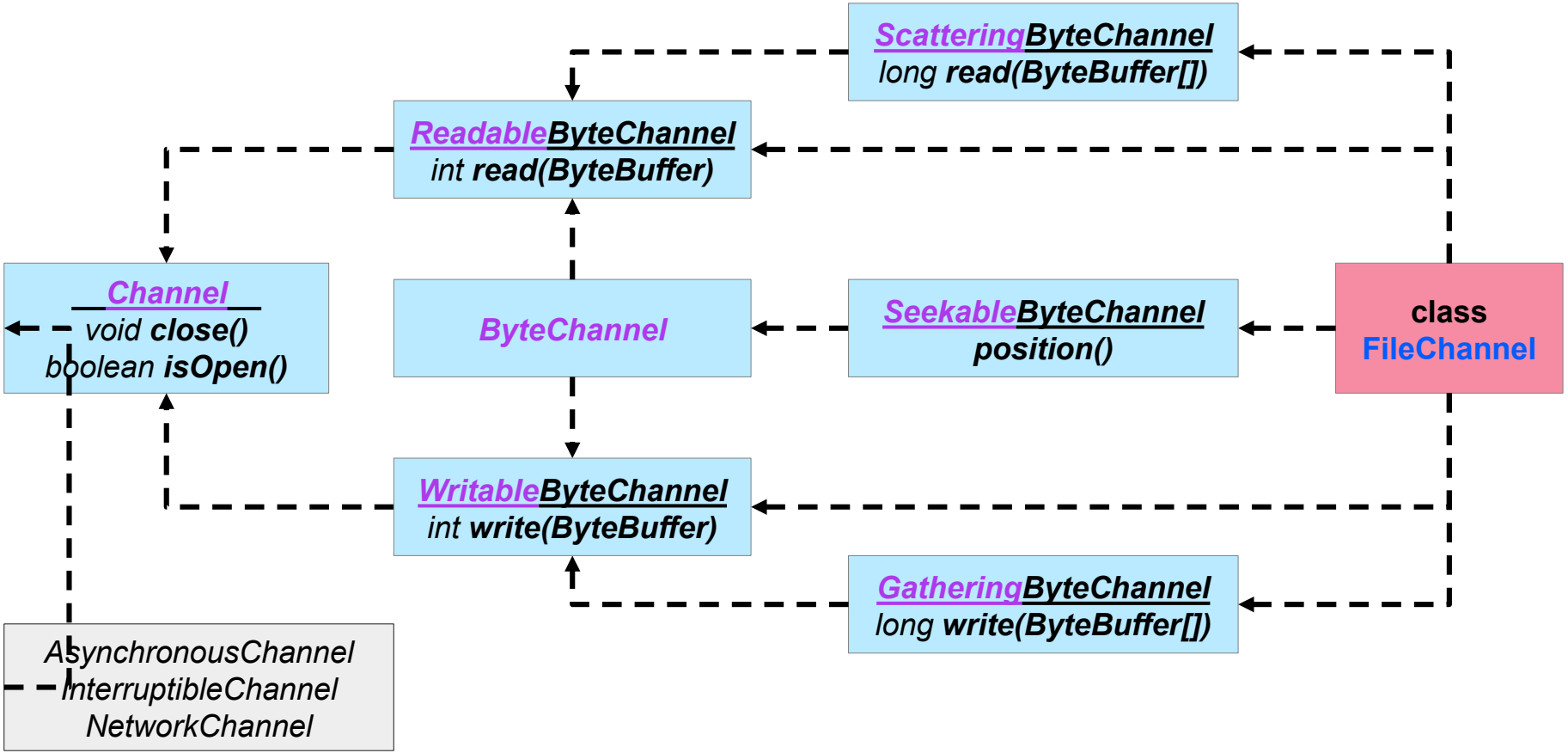
Каналы

ИТМО

- java.nio.channels
- **Файловые каналы** и сетевые каналы
- Отличие от потоков
  - поддержка двунаправленного ввода-вывода
  - поддержка неблокирующего ввода-вывода
  - поддержка асинхронного ввода-вывода
  - чтение и запись целого буфера

Java

Каналы - интерфейсы и FileChannel

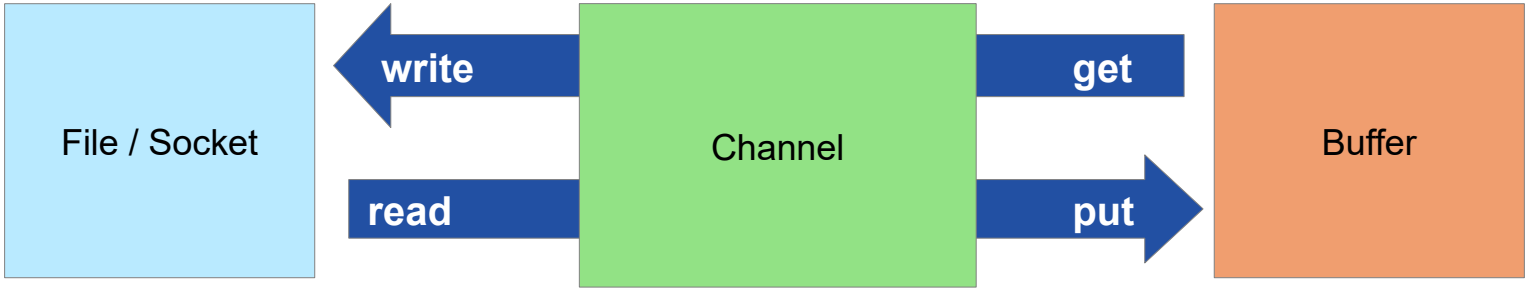


☑ Канал FileChannel

ИТМО

- FileChannel.open(path, options)
  - enum StandardOpenOptions
  - READ, WRITE, APPEND, CREATE
- FileInputStream.getChannel()
- FileOutputStream.getChannel()

- write(ByteBuffer b)
  - берем из буфера (get)
  - пишем в файл (write)
- read(ByteBuffer b)
  - читаем из файла (read)
  - кладем в буфер (put)



Java

Чтение и запись

ИТМО

```
ByteBuffer buffer = ByteBuffer.allocate(1000);
```

- Чтение из файла (файлового канала)

```
Path path = Paths.get("in.txt");
FileChannel channel = FileChannel.open(path, READ);
buffer.clear();
int nBytes = channel.read(buffer);
```

- Запись в файл (файловый канал)

```
Path path = Paths.get("out.txt");
FileChannel channel = FileChannel.open(path, WRITE,CREATE);
buffer.flip();
int nBytes = channel.write(buffer);
```

Java

Продвинутые каналы

ИТМО

- Передача данных из канала в канал

```
transferFrom(ReadableByteChannel, long position, long count)
transferTo(long position, long count, WritableByteChannel)
```

- ScatteringByteChannel / GatheringByteChannel

```
ByteBuffer bufferArray = new ByteBuffer[3];
for (ByteBuffer buf : bufferArray) { buf.allocate(256); }
scatteringChannel.read(bufferArray);
gatheringChannel.write(bufferArray);
```

Java

Отображение файла в память

ИТМО

- FileChannel
  - map() - получение MappedByteBuffer
  - отображение файла в память
- MappedByteBuffer
  - boolean isLoaded()
  - load()
  - force()

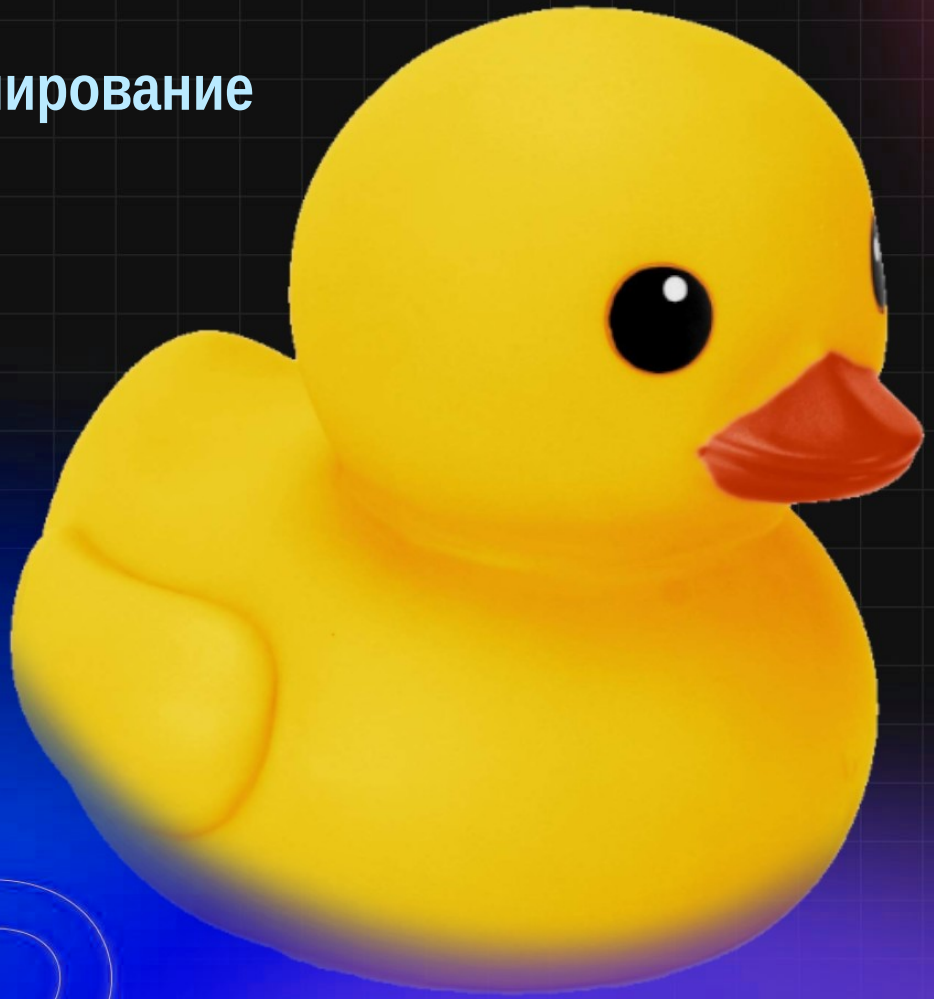
```
var mode = FileChannel.MapMode.READ_ONLY;
var buffer =
    channel.map(mode, 0, fileChannel.size());
```

Java

Программирование

2 семестр

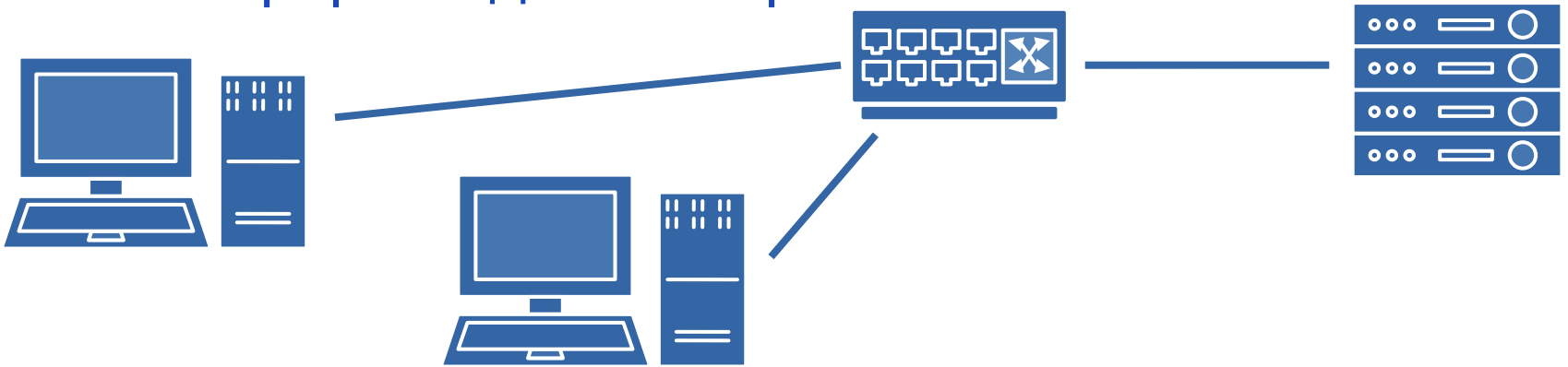
2026



Сетевое взаимодействие

Терминология

- *Вычислительная сеть* — система для передачи данных между *узлами сети*.
- *Хост* — компьютер, подключенный к сети и имеющий сетевой адрес.
- *Протокол* — набор правил, определяющих порядок действий и формат данных при сетевом обмене.



Сетевая архитектура

ИТМО

- *Клиент-серверная* архитектура
  - Централизованное управление и обмен данными
  - Сервер предоставляет сервисы в режиме ожидания запроса
  - Клиент получает результат от сервера по запросу
  - Надежность зависит от сервера — критический узел
- *Одноранговая* архитектура (пиринговая)
  - Децентрализованное управление и обмен данными
  - Все узлы (pears) равноправны, могут быть клиентом и сервером
  - Нет критического узла

Java

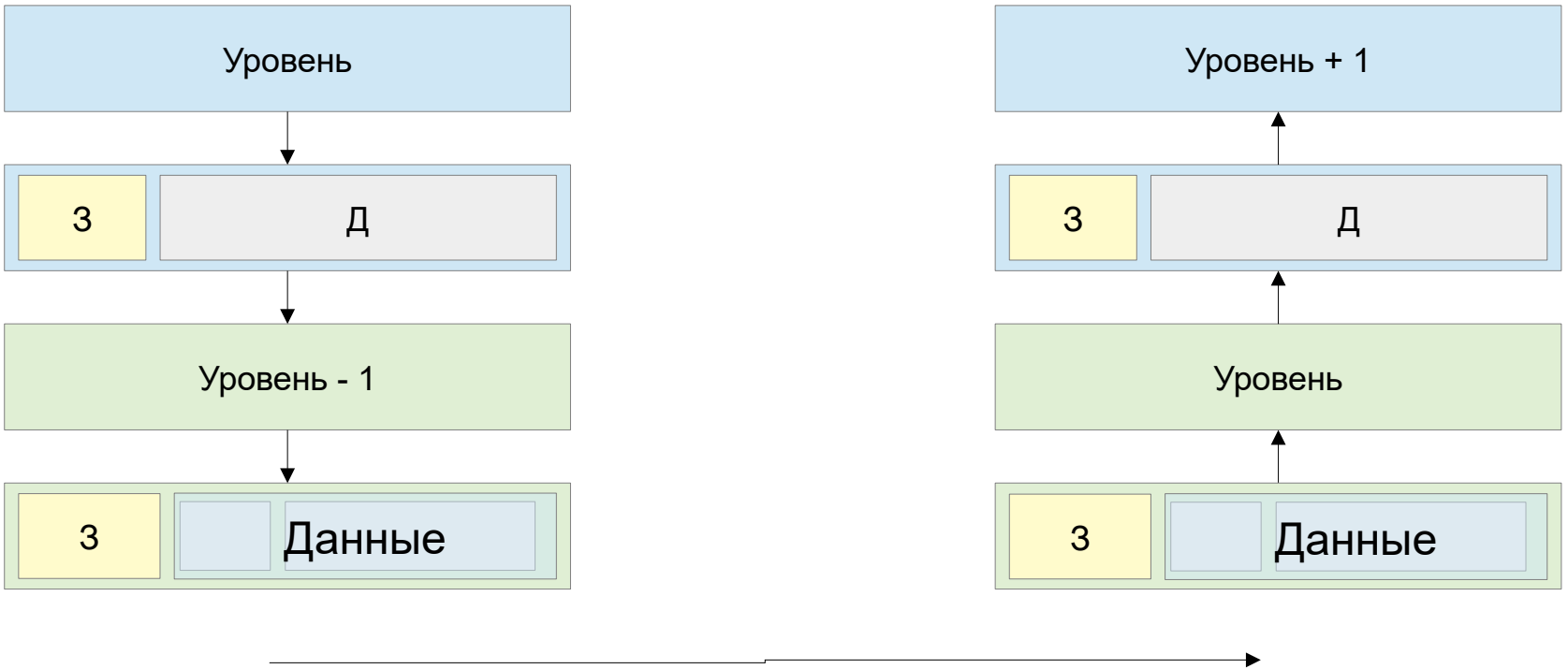
Модель передачи данных

- модель ISO/OSI
- стек TCP/IP

TCP/IP	OSI	Пример
Прикладной	Прикладной	HTTP
	Представительский	
	Сеансовый	
Транспортный	Транспортный	TCP, UDP
Сетевой	Сетевой	IP
Канальный	Канальный	Ethernet
	Физический	витая пара

Пакеты

- Пакет = заголовок + данные



Сетевое взаимодействие в Java

ИТМО

- Протокол прикладного уровня - HTTP

- Протоколы транспортного уровня
- TCP
  - устанавливается соединение
  - подтверждение доставки
  - надежность передачи данных
- UDP
  - без установление соединения
  - без подтверждения доставки
  - скорость передачи данных

Java

IP-адрес

- Идентифицирует связь между роутером и хостом
- ID сети (префикс) + ID хоста (суффикс)
- IPv4 — 32 бита (194.85.160.55)
  - Класс A: префикс 8 бит (0...) + суффикс 24 бита
  - Класс B: префикс 16 бит (10…) + суффикс 16 бит
  - Класс C: префикс 24 бита (110…) + суффикс 8 бит
  - Маска подсети: 192.85.160.55
    - 192.168.0.5/255.255.255.240
    - 192.168.0.5/28
- IPv6 — 128 бит [FC05::4429:0:AC02]
- Loopback (localhost - 127.0.0.1 / [::1]

DNS

- DNS — Domain Name Service
- Преобразование между доменным именем и IP-адресом
- www.google.com ↔ 172.217.23.132

Порт

- IP-адрес идентифицирует хост
- порт идентифицирует процесс (приложение)
- *Сокет* — интерфейс для обмена — адрес и порт

- Для обмена данными нужно знать:
  - протокол
  - IP-адрес и порт отправителя
  - IP-адрес и порт получателя

InetAddress

- Класс InetAddress
  - Inet4Address Inet6Address
- Методы InetAddress (статические)
  - InetAddress getLocalHost()
  - InetAddress getByAddress(byte[] addr)
  - InetAddress getByName(String name) — обращение к DNS
- Нестатические методы
  - byte[] getAddress
  - String getHostName()

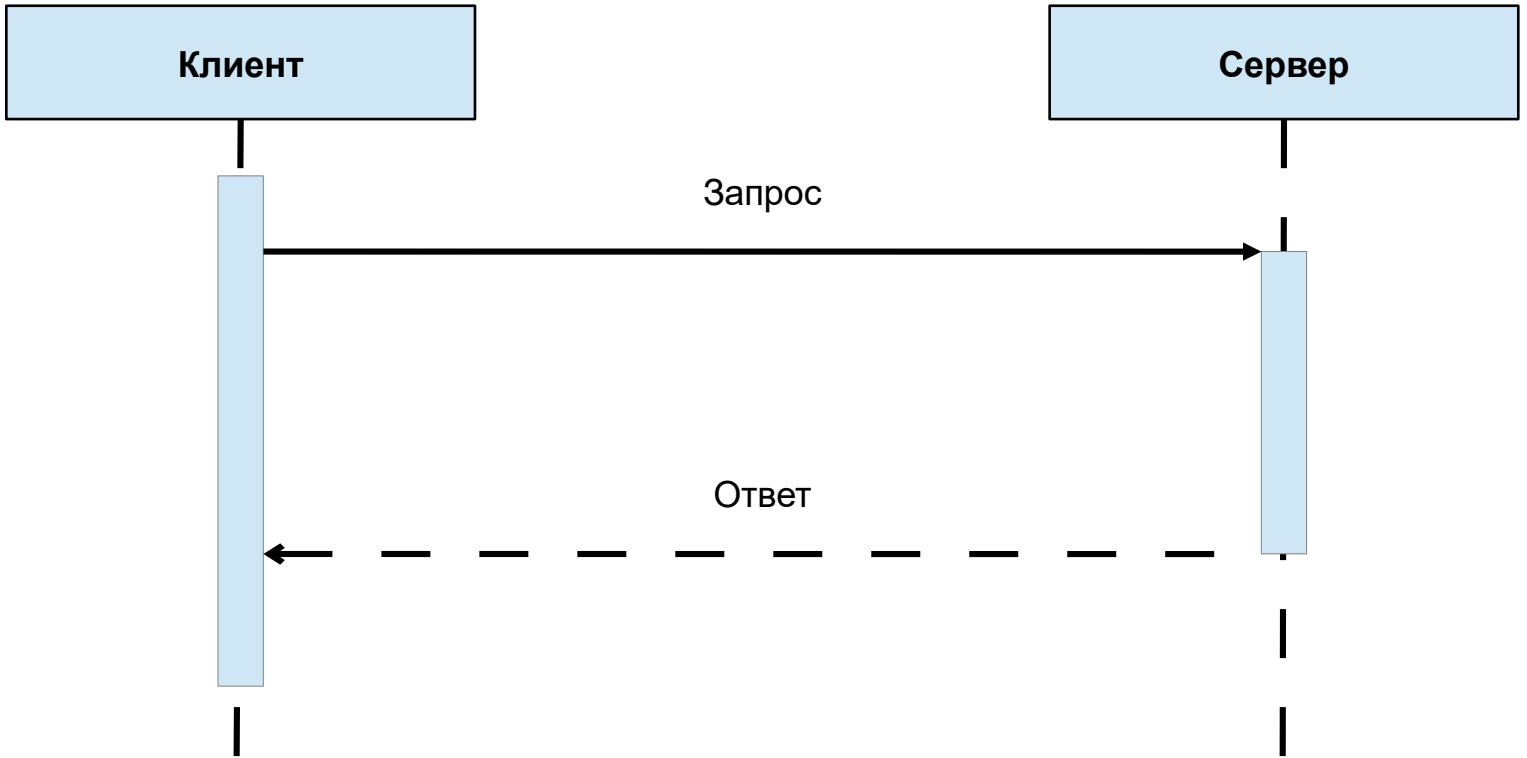
InetSocketAddress

- InetSocketAddress(InetAddress addr, int port)
- InetSocketAddress(int port)
- InetSocketAddress(String hostname, int port)

Сетевой обмен

- Клиент
  - Работает на любом хосте (сервер не знает, где именно)
  - Свободный порт выбирается при отправлении запроса
  - Посылает запрос серверу, ждет ответ
- Запрос
  - Содержит данные и информацию о клиенте
- Сервер
  - Работает на известном хосте (известный IP-адрес)
  - Прослушивает известный порт (зависит от сервиса)
  - Ждет запрос от клиента, посылает ответ
- Ответ
  - Содержит данные

Диаграмма последовательностей



Обмен по протоколу UDP

ИТМО

- java.net.DatagramPacket — датаграмма (передаваемые данные + служебная информация)
  - Адрес буфера
  - Длина буфера (не более 64 Кбайт)
  - Адрес получателя (при отправлении)
- java.net.DatagramSocket — сокет для обмена
  - Порт для прослушивания (для получения)
  - Адрес и порт (для отправления)

Java

Пример обмена по протоколу UDP

ИТМО

```
// клиент

byte arr[] = {0,1,2,3,4,5,6,7,8,9};
int len = arr.length;
DatagramSocket ds; DatagramPacket dp;
InetAddress host; int port;

ds = new  DatagramSocket();

host = InetAddress.get...();
port = 6789;
dp = new DatagramPacket(arr,len,host,port);
ds.send(dp);

dp = new DatagramPacket(arr,len);
ds.receive(dp);

for (byte j : arr) {
    System.out.println(j);
}
```

```
// сервер

byte arr[] = new byte[10];
int len = arr.length;
DatagramSocket ds; DatagramPacket dp;
InetAddress host; int port = 6789;

ds = new DatagramSocket(port);

dp = new DatagramPacket(arr,len);
ds.receive(dp);

for (int j = 0; j < len; j++) {
    arr[j] *= 2;
}

host = dp.getAddress();
port = dp.getPort();
dp = new DatagramPacket(arr,len,host,port);
ds.send(dp);
```

Java

Обмен по протоколу TCP

ИТМО

- java.net.Socket — сокет для обмена (клиент и сервер)
  - new Socket (адрес + порт — для отправления)
  - Socket ServerSocket.accept() - для получения
- java.net.ServerSocket — фабрика сокетов
  - new ServerSocket(порт) — на стороне сервера
- обмен данными через потоки ввода-вывода
  - Socket.getInputStream()
  - Socket.getOutputStream()

Java

Пример обмена по протоколу TCP

ИТМО

```
// клиент

byte arr[] = {0,1,2,3,4,5,6,7,8,9};
int len = arr.length;
Socket sock;
OutputStream os; InputStream is;
InetAddress host; int port;

port = 6789;
sock = new Socket(host,port);

os = sock.getOutputStream();
os.write(arr);

is = sock.getInputStream();
is.read(arr);

for (byte j : arr) {
    System.out.println(j);
}
```

```
// сервер

byte arr[] = new byte[10];
int len = arr.length;
Socket sock; ServerSocket serv;
OutputStream os; InputStream is;
InetAddress host; int port = 6789;

serv = new ServerSocket(port);
sock = serv.accept();

is = sock.getInputStream();
is.read(arr);

for (int j = 0; j < len; j++) {
    arr[j] *= 2;
}

os = sock.getOutputStream();
os.write(arr);
```

Java

Использование каналов и java.nio

ИТМО

- Протокол UDP — DatagramChannel
  - DatagramChannel open()
  - bind(SocketAddress local) // сервер
  - SocketAddress receive(ByteBuffer)
  - int send(ByteBuffer, SocketAddress)

  - connect(SocketAddress remote) // клиент
  - int read(ByteBuffer)
  - int write(ByteBuffer)

Java

Пример обмена по протоколу UDP (NIO)

ИТМО

```
// клиент

byte arr[] = {0,1,2,3,4,5,6,7,8,9};
int len = b.length;
DatagramChannel dc; ByteBuffer buf;
InetAddress host; int port;
SocketAddress addr;

addr = new InetSocketAddress(host,port);
dc = DatagramChannel.open();

buf = ByteBuffer.wrap(arr);
dc.send(buf, addr);

buf.clear();
addr = dc.receive(buf);

for (byte j : arr) {
    System.out.println(j);
}
```

```
// сервер

byte arr[] = new byte[10];
int len = arr.length;
DatagramChannel dc; ByteBuffer buf;
InetAddress host; int port = 6789;
SocketAddress addr;

addr = new InetSocketAddress(port);
dc = DatagramChannel.open();
dc.bind(addr);

buf = ByteBuffer.wrap(arr);
addr = dc.receive(buf);

for (int j = 0; j < len; j++) {
    arr[j] *= 2;
}

buf.flip();
dc.send(buf, addr);
```

Java

Использование каналов и java.nio

ИТМО

- Протокол TCP
  - ServerSocketChannel
    - ServerSocketChannel open()
    - bind(SocketAddress local)
    - SocketChannel accept() // сервер
  - SocketChannel
    - SocketChannel connect(SocketAddress remote) / клиент
    - write(ByteBuffer)
    - read(ByteBuffer)

Java

Пример обмена по протоколу TCP (NIO)

ИТМО

```java
// клиент

byte arr[] = {0,1,2,3,4,5,6,7,8,9};
int len = arr.length;
InetAddress host; int port;
SocketAddress addr; SocketChannel sock;

port = 6789;
addr = new InetSocketAddress(host,port);
sock = SocketChannel.open();
sock.connect(addr);

buf = ByteBuffer.wrap(arr);
sock.write(buf);

buf.clear();
sock.read(buf);

for (byte j : arr) {
    System.out.println(j);
}
```

```java
// сервер

byte arr[] = new byte[10];
int len = arr.length;
InetAddress host; int port = 6789;
SocketAddress addr; SocketChannel sock;
ServerSocketChannel serv;

serv = ServerSocketChannel.open();
serv.bind(port);
sock = serv.accept();

buf = ByteBuffer.wrap(arr);
sock.read(buf);

for (int j = 0; j < len; j++) {
    arr[j] *= 2;
}

buf.flip();
sock.write(buf);
```

Java

Режимы обмена

- Блокирующий режим
  - Можно ли выполнить операцию = попытаться выполнить
    - setSoTimeout(long milliseconds)
    - Socket / ServerSocket / DatagramSocket
- Неблокирующий режим
  - Проверка возможности - отдельно от самой операции

Неблокирующий ввод-вывод

ИТМО

- ServerSocketChannel.configureBlocking(false)
  - Метод accept() возвращает SocketChannel или null, если соединение не установлено
- SocketChannel.configureBlocking(false)
  - Методы read и write возвращают int — количество прочитанных байт, или -1, если данных больше нет.

Java

Селекторы

ИТМО

- abstract class Selector
  - open()
  - select()
  - Set<SelectionKey> keys() // key set
  - Set<SelectionKey> selectionKeys() // selected key set
  - // cancelled key set

Java

Селекторы - ключи

ИТМО

- abstract class SelectionKey
  - interestOps(), readyOps()
  - OP_CONNECT, OP_ACCEPT, OP_READ, OP_WRITE
  - isConnectable(), isAcceptable(), isReadable(), isWriteable()
  - channel(), selector()
  - attach(Object), Object attachment()
  - cancel()

Java

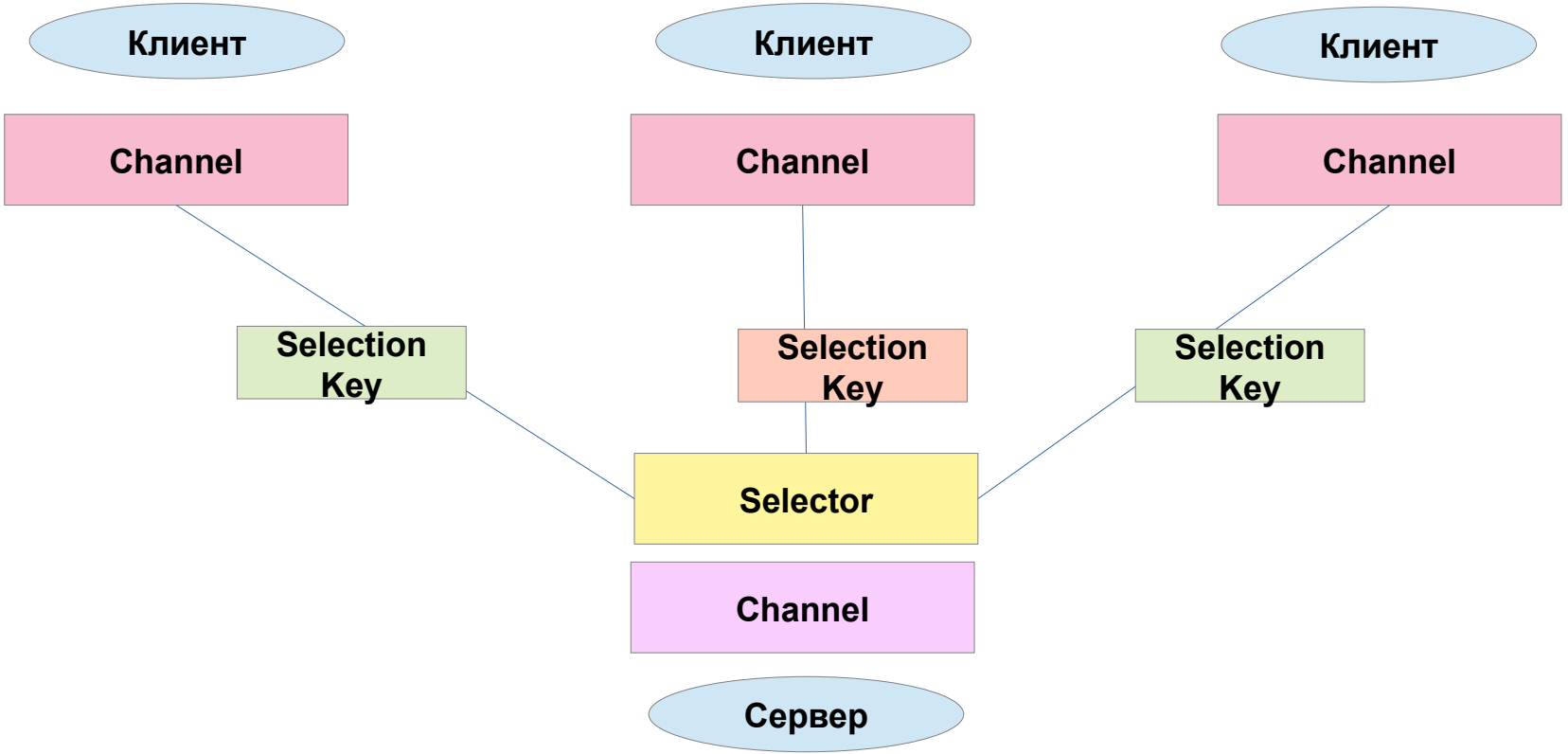
Селекторы - каналы

ИТМО

- abstract class SelectableChannel
  - SelectonKey register(Selector s, int Ops, Object attachment)
    - SocketChannel
    - ServerSocketChannel
    - DatagramChannel

Java

Селекторы и каналы



Пример

```
Selector selector = Selector.open();
ServerSocketChannel server = ServerSocketChannel.open();
server.configureBlocking(false);
server.register(selector, SelectionKey.OP_ACCEPT);
while(true) {
    selector.select();
    Set<SelectionKey> keys = selector.selectedKeys();
    for (var iter = keys.iterator(); iter.hasNext(); ) {
        SelectionKey key = iter.next(); iter.remove();
        if (key.isValid()) {
            if (key.isAcceptable()) { doAccept(); }
            if (key.isReadable()) { doRead(); }
            if (key.isWritable()) { doWrite(); }
        }
    }
}
selector.close();
```

accept, read, write

ИТМО

```
doAccept() {
  var ssc = (ServerSocketChannel) key.channel();
  var sc = ssc.accept();
  key.attach(clientData);
  sc.configureBlocking(false);
  sc.register(key.selector(), OP_READ);
}
```

```
doRead() {
  var sc = (SocketChannel) key.channel();
  var data = (ClientData) key.attachment();
  sc.read(data.buffer);
  sc.register(key.selector(), OP_WRITE);
}
```

```
doWrite() {
  var sc = (SocketChannel) key.channel();
  var data = (ClientData) key.attachment();
  sc.write(data.buffer);
}
```

Java

Протокол HTTP

- URI — Unified Resource Identifier
  - URL — Unified Resource Locator
  - URN — Unified Resource Name

Класс URL

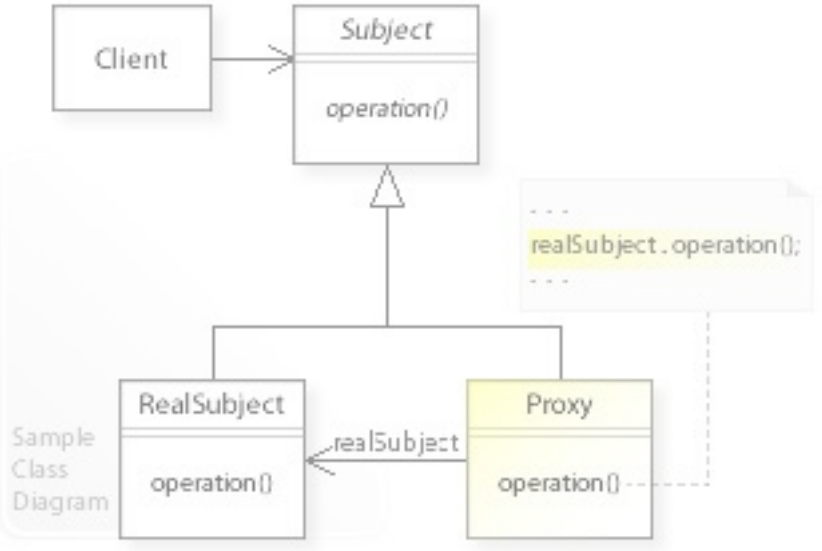
```
URL url = new URL("http://www.google.com");

InputStream is = url.openStream();
is.read();
is.close();

Object o = url.getContent();
```

Класс URLConnection

```
URL url = new URL("http://www.google.com");
URLConnection uc = url.getConnection();
uc.connect();

InputStream is = uc.getInputStream();
// is.read();

uc.setDoOutput(true);
OutputStream os = uc.getOutputStream();
// os.write();

is.close();
os.close();
uc.close()
```

Шаблон Proxy

- Заместитель - делегирование **с тем же интерфейсом**
- Подмена реального объекта заместителем
  - Virtual - ленивое создание
  - Remote - удаленный объект
  - Security - контроль доступа
  - Caching - кэширование
  - Smart - логирование
- java.lang.reflect.Proxy

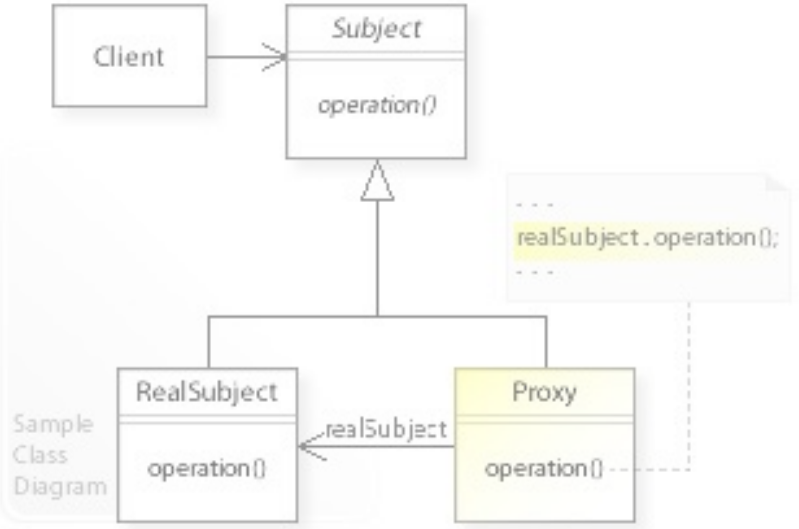


Proxy

- Заместитель

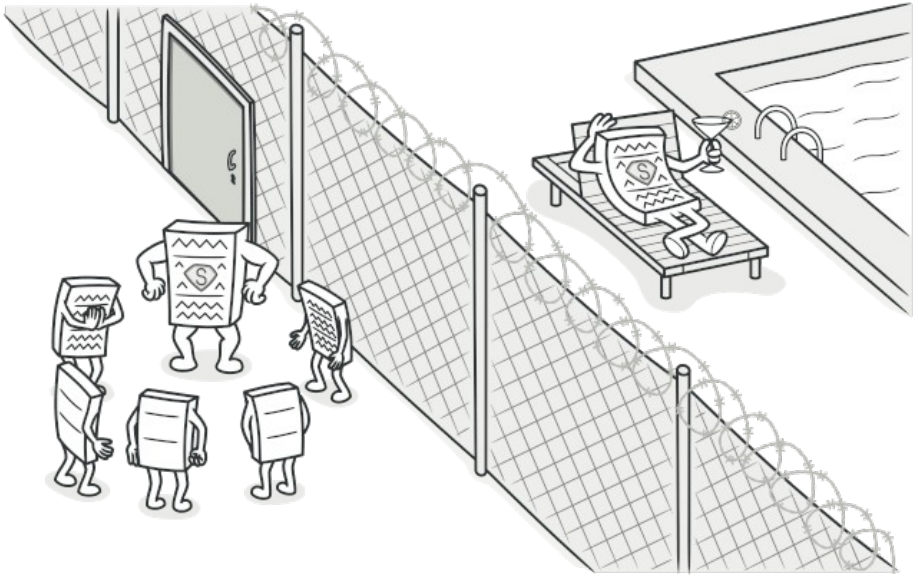
```
class Proxy implements Subject {
  RealSubject real;
  public request() {
    return real.request();
  }
}
```

```
Subject subject = new Proxy();
subject.request();
```



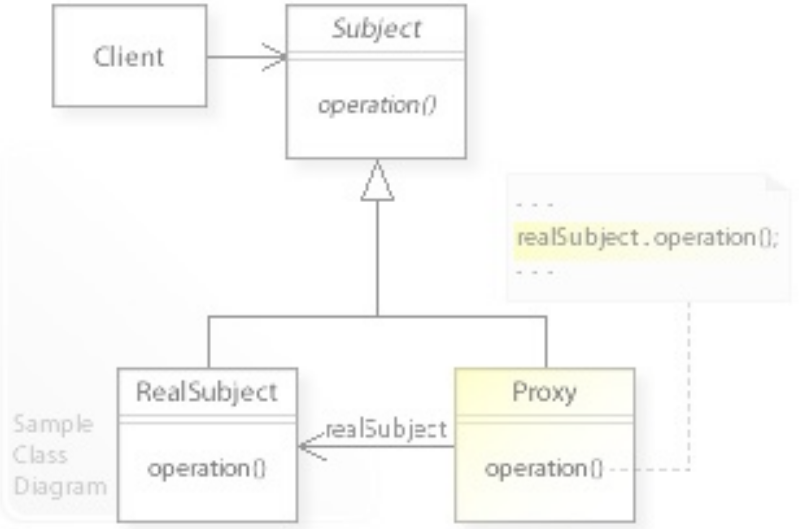
Proxy

- Контроль работы с реальным объектом
- Клиент не знает, используется прокси или нет
- Замедление и усложнение доступа



Remote Proxy

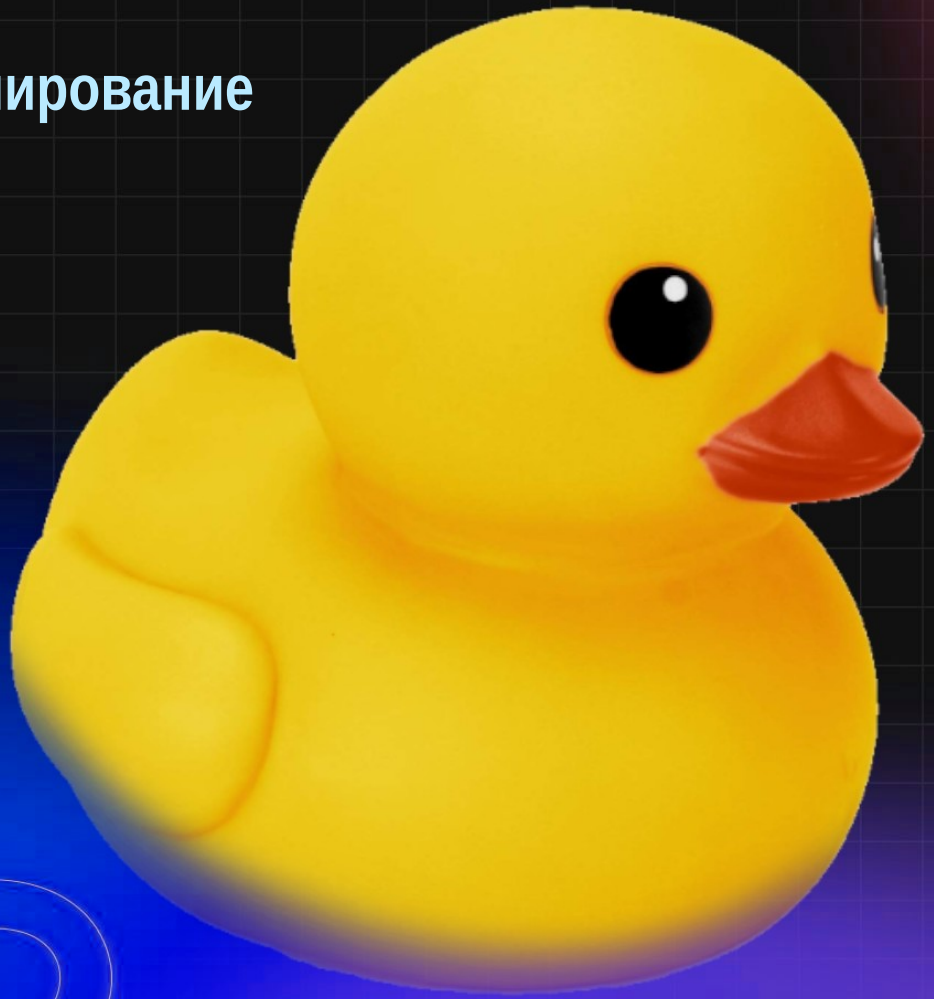
- Идея на лабу 6
  - interface Subject - методы из CollectionManager
  - RealCollManager - на сервере
  - ProxyCollManager - на клиенте



Программирование

2 семестр

2026



Дата и время
Date/Time API

Использование даты и времени

ИТМО

- Дата и время — 2 варианта представления:
  - Человеческое время — часы, минуты, дни, недели, месяцы
  - Машинное время — миллисекунды от точки отсчета
    - EPOCH - 1 января 1970 года, 00:00:00 (Java, UNIX)
    - Windows - 1 января 1601 года
  - Y2K problem (2000)
  - Y 2038 problem
    - 19-01-2038 03:14:07 + 1 sec = 13-12-1901 20:45:52

Java

Традиционные классы. Date

ИТМО

- Date 1.0
  - все действия с датой
  - человеческое и машинное представление
  - форматирование даты
- Конструкторы
  - Date
  - Date(long)
- Date 1.1
  - только момент времени
  - почти все методы - deprecated
- Методы
  - long getTime()
  - boolean after(Date)
  - boolean before(Date)

Java

TimeZone

- Временная зона — смещение от стандартного:
- до 1972 года - Гринвич (GMT)
- после 1972 — UTC — всемирное координированное
- Методы
  - getDefault()
  - getAvailableIDs()
  - getRawOffset() - смещение без учета летнего времени
  - getOffset(long date) — с учетом летнего времени
- Класс SimpleTimeZone — реализованный потомок

Calendar

- Абстрактный класс — машинное ↔ человеческое
  - Calendar getInstance()
  - add(int field, int amount);
  - roll(int field, int amount);
  - set(int field, int value);
  - Date getTime()
  - setTime(Date)
- класс GregorianCalendar
  - сочетает 2 календаря (григорианский и юлианский)

Date/Time API - новые классы

ИТМО

- java.time - дата, время, периоды
  - LocalDate, LocalTime, LocalDateTime,
  - Instant, Duration, Period,
  - OffsetTime, OffsetDateTime, ZonedDateTime
- java.time.chrono - календарные системы
- java.time.format - форматирование даты и времени
- java.time.temporal - единицы измерения и отдельные поля
- java.time.zone - временные зоны и правила

Java

Дни недели и месяцы (enums)

ИТМО

- enum DayOfWeek (1 (MONDAY) — 7 (SUNDAY))
- enum Month (1 (JANUARY) — 12 (DECEMBER))
- метод getDisplayName(style, locale)
- стиль — FULL, NARROW, SHORT / STANDALONE

Java

Представление даты и времени

- Year
- YearMonth
- MonthDay
- LocalDate
- LocalTime
- LocalDateTime

2026-03-10 T 16:43:58.355048492

Соглашения по именам методов

ИТМО

- Статические
  - of — создает экземпляр на основе входных параметров
    - LocalDate.of(year, month, day), ofYearDay(year, dayOfYear)
  - from — конвертирует экземпляр из другого типа
    - LocalDate.from(LocalDateTime)
  - parse — создает экземпляр из строкового представления
    - LocalDate.parse("2022-02-22")

Java

Соглашения по именам методов

ИТМО

- Методы экземпляра
  - format — форматирует объект в строку
  - get — возвращает поля объекта // getHours()
  - with — возвращает копию с изменением // withYear(2021)
  - plus — возвращает копию с добавлением // plusDays(2)
  - minus — возвращает копию с убавлением // minusWeeks(3)
  - to — преобразует объект в другой тип // toLocalTime()
  - at — комбинирует объект с другим // date.atTime(time)

Java

Момент времени

- Класс Instant (timestamp)
  - now()
  - plusNanos()
  - plusMillis()
  - plusSeconds()
  - minusNanos()
  - minusMillis()
  - minusSeconds()

Временная зона

- ZoneOffset — разница со стандартным временем
  - UTC+01:00, GMT-2
- ZoneId — идентификатор зоны
  - Europe/Moscow
- OffsetTime = LocalTime + ZoneOffset
- OffsetDateTime = LocalDateTime + ZoneOffset
- ZonedDateTime = LocalDateTime + ZoneId
  - использует java.time.zone.ZoneRules

Когда использовать классы

ИТМО

- YearMonth - срок действия банковской карты
- MonthDay - праздники (повторяются каждый год)
- LocalDate - местная дата
- LocalDateTime - местные дата и время
- LocalTime - местное время
- OffsetTime - поясное время (веб, базы данных)
- ZonedDateTime - полное зональное время

Java

Разбор и форматирование

ИТМО

- java.time.format.DateTimeFormatter
  - константы формата:
    - BASIC_ISO_DATE ('20240229')
    - ISO_LOCAL_DATE ('2024-02-29')
    - ISO_ZONED_DATETIME ('2024-02-29T18:15:30+03:00[Europe/Moscow]')
    - RFC_1123_DATE_TIME ('Thu, 29 Feb 2024 18:15:30 GMT+3')
  - локализованные форматы
    - ofLocalizedDateTime(dateStyle, timeStyle)
    - стиль (FULL, LONG, MEDIUM, SHORT)
  - шаблон
    - ofPattern()
  - методы format() и parse()

Java

Периоды даты и времени

ИТМО

- Duration — продолжительность в часах и менее
  - toNanos(), toMillis(), toSeconds(), toMinutes(), toHours(), toDays()
- Period — период в днях и более
  - getDays(), getMonths(), getYears()
- .between()
- .plus
- .minus

Java

Старые и новые классы

ИТМО

- Соответствия:
  - java.util.Date — java.time.Instant
  - java.util.GregorianCalendar — java.time.ZonedDateTime
  - java.util.TimeZone — ZoneId, ZoneOffset
- Методы:
  - Calendar.toInstant()
  - GregorianCalendar.toZonedDateTime()
  - GregorianCalendar.fromZonedDateTime()
  - Date.fromInstant()
  - Date.toInstant()
  - TimeZone.toZoneId()

Java

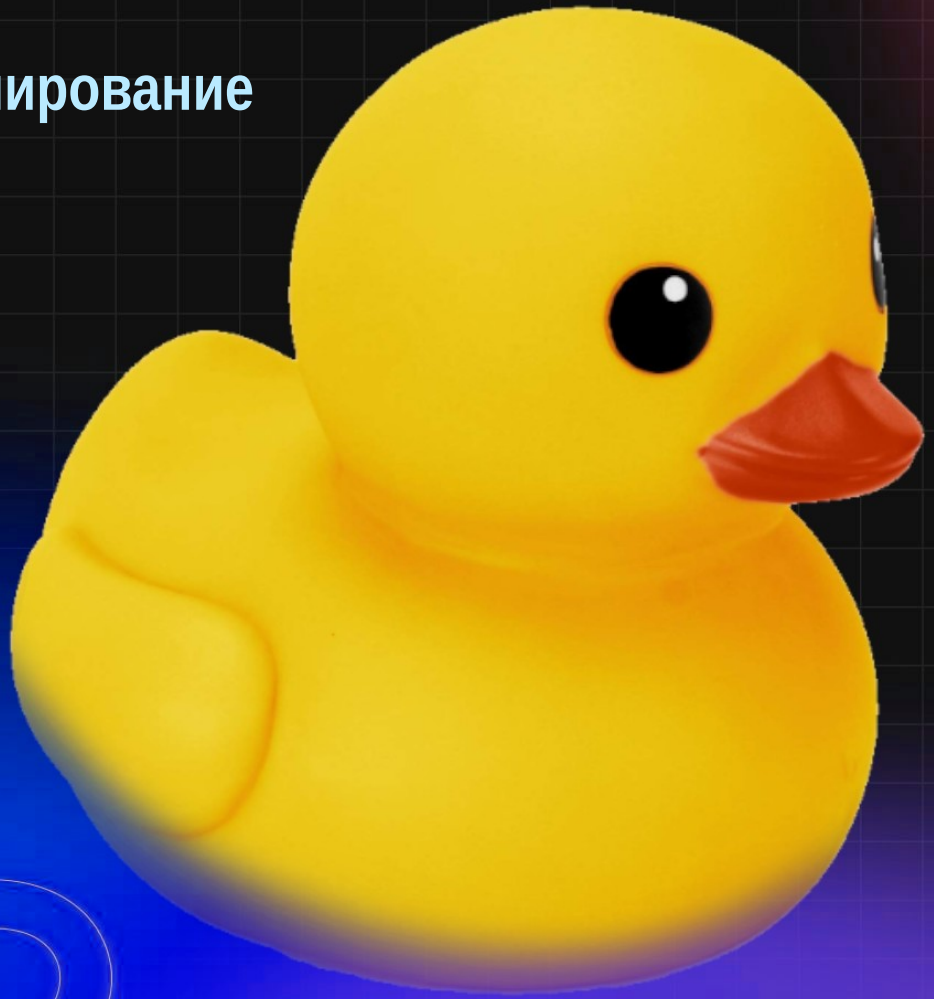
Примеры

```
var date1 = LocalDate.of(2026, 03, 10);
var date2 = LocalDate.parse("2026-01-01");
var date3 = LocalDate.now()
                          .with(TemporalAdjusters.firstDayOfMonth());

var dateTime1 = LocalDateTime.now();
var dateTime2 = dateTime1.minusHours(2).plusMinutes(30);

var zoneId = ZoneId.of("Europe/Moscow");
var zonedDateTime = ZonedDateTime.of(dateTime1, zoneId);

var period = Period.between(date1, date2);
var duration = Duration.between(dateTime1, dateTime2);
```

Программирование

2 семестр

2026



Функциональное программирование

Функциональное программирование

ИТМО

- Функции высшего порядка
  - могут быть аргументами и возвращаемыми значениями
- Ленивые вычисления
- Нет побочных эффектов
- Нет состояния

- Достоинства:
  - Проще тестирование
  - Проще распараллеливание
  - Оптимизация кода

Java

Итерация и рекурсия

ИТМО

- Итерация → Рекурсия
- Проблема рекурсии — ограничение стека
  - Вызов функции — параметры и адрес возврата — в стек
  - Во время работы функции локальные переменные в стеке
  - Возврат — очистка стека и переход по адресу возврата

Java

Итерация и рекурсия

ИТМО

- Итерация → Рекурсия
- Проблема рекурсии — ограничение стека
  - Вызов функции — параметры и адрес возврата — в стек
  - Во время работы функции локальные переменные в стеке
  - Возврат — очистка стека и переход по адресу возврата
- Решение — хвостовая рекурсия
  - Рекурсивный вызов функции — последняя команда
  - Вместо повторных рекурсивных вызовов — замена параметров и возврат к началу (фактически — итерация)

Java

Факториал (итерация)

- Итерация

```
public int factor(int n) {
  int result = 1;
  int i = 1
  while (i <= n) {
    result *= i;
    i += 1;
  }
  return result;
}
```

Факториал (рекурсия)

ИТМО

- Итерация

```
public int factor(int n) {
  int result = 1;
  int i = 1;
  while (i <= n) {
    result *= i;
    i += 1;
  }
  return result;
}
```

- Рекурсия
  - проще код
  - проблема стека вызовов

```
public int factor(int n) {
  return n <= 1 ?
         1 :
         factor(n - 1) * n;
}
```

Java

Стек вызовов при рекурсии

ИТМО

- При вызове метода - в стек помещаются параметры и адрес возврата
- При работе метода - в стек помещаются локальные переменные
- Перед возвратом - очистка локальных переменных
- Во время возврата - очистка от параметров и возврат

адрес возврата 1
int n

```
public int factor(int n) {
  return n <= 1 ?
         1 :
         factor(n – 1) * n;
}
```

Стек вызовов при рекурсии

ИТМО

- При вызове метода - в стек помещаются параметры и адрес возврата
- При работе метода - в стек помещаются локальные переменные
- Перед возвратом - очистка локальных переменных
- Во время возврата - очистка от параметров и возврат

адрес возврата 1
int n
адрес возврата 2
int n

```
public int factor(int n) {
  return n <= 1 ?
         1 :
         factor(n – 1) * n;
}
```

Стек вызовов при рекурсии

ИТМО

- При вызове метода - в стек помещаются параметры и адрес возврата
- При работе метода - в стек помещаются локальные переменные
- Перед возвратом - очистка локальных переменных
- Во время возврата - очистка от параметров и возврат

адрес возврата 1
int n
адрес возврата 2
int n
адрес возврата 3
int n

```
public int factor(int n) {
  return n <= 1 ?
         1 :
         factor(n – 1) * n;
}
```

Java

Стек вызовов при рекурсии

ИТМО

- При вызове метода - в стек помещаются параметры и адрес возврата
- При работе метода - в стек помещаются локальные переменные
- Перед возвратом - очистка локальных переменных
- Во время возврата - очистка от параметров и возврат

адрес возврата 1
int n
адрес возврата 2
int n

```
public int factor(int n) {
  return n <= 1 ?
         1 :
         factor(n – 1) * n;
}
```

Факториал (хвостовая рекурсия)

ИТМО

- Хвостовая рекурсия

```
public int factor(int n, int p) {
  return n <= 1 ?
         p :
         factor(n – 1, p * n);
  }
}

public int factor(int n) {
  return factor(n, 1);
}
```

- Обычная рекурсия

```
public int factor(int n) {
  return n <= 1 ?
         1 :
         factor(n – 1) * n;
}
```

Java

Факториал (хвостовая рекурсия)

ИТМО

- Хвостовая рекурсия

```
public int factor(int n, int p) {
  return n <= 1 ?
         p :
         factor(n – 1, p * n);
  }
}

public int factor(int n) {
  return factor(n, 1);
}
```

адрес возврата 1
int n
int p

Java

Факториал (хвостовая рекурсия)

ИТМО

- Хвостовая рекурсия

```
public int factor(int n, int p) {
  return n <= 1 ?
         p :
         factor(n – 1, p * n);
  }
}

public int factor(int n) {
  return factor(n, 1);
}
```

адрес возврата 1
адрес возврата 2
int n
int p

Java

Стек при хвостовой рекурсии

ИТМО

- Хвостовая рекурсия

```
public int factor(int n, int p) {
  return n <= 1 ?
         p :
         factor(n – 1, p * n);
  }
}

public int factor(int n) {
  return factor(n, 1);
}
```

адрес возврата 1
int n
int p

Java

Элементы λ-исчисления

ИТМО

- Алонзо Чёрч (Alonzo Church)
- â.a+1 → ʌa.a+1 → Λa.a+1 → λa.a+1
  - ❖ â — аргумент выражения a+1
- Переменная: x
- Операции:
  - ❖ Абстракция: λx.f (связывание x с функцией f)
  - ❖ Аппликация (применение): f g (применение f к аргументу g)

Пример

- inc(x) = x + 1
- f(x) = x + 1
- (x) → x + 1
- λx.x+1

inc(3)

f(3)

((x) → x + 1) (3)

- Свободные и связанные переменные

λ x . x + y

связанная

свободная

Функции обратного вызова (callback)

ИТМО

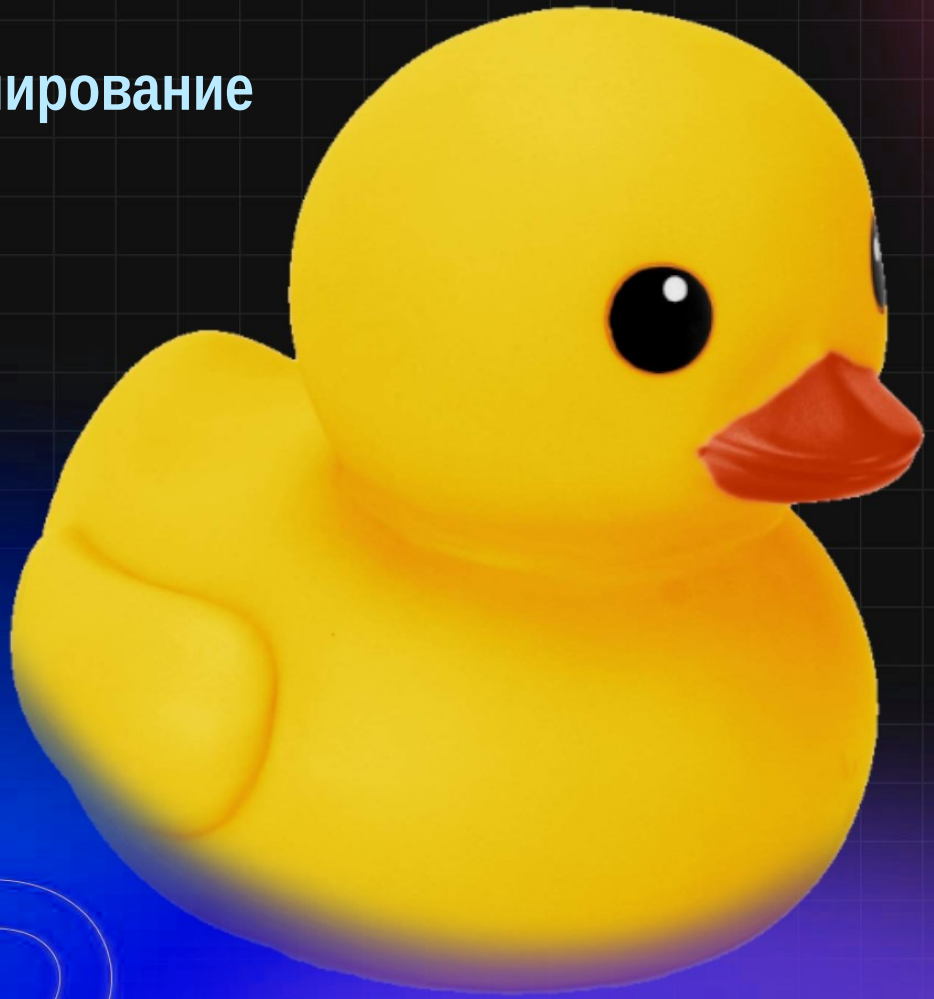
- Передача функции с целью ее дальнейшего вызова
  - реализация действия разными способами, выбираемыми во время исполнения
  - реализация асинхронной реакции на события
- Варианты реализации
  - указатели на функцию (C, C++)
  - делегаты (C#)
  - объект интерфейса с методом / анонимный класс (Java < 8)
  - λ-выражения (Java 8+)

Java

Программирование

2 семестр

2026



Практические примеры

Пример - список студентов

ИТМО

```
class Student {

  public String getName() { ... }
  public double getAge() { ... }
  public double getAvgMark() { ... }
  public String getGroup() { ... }
  public String getEmail() { ... }

  static List<Student> students;

  public static void printAll() {
    for (Student st : students) {
      System.out.println(st.getName());
    }
  }
}
```

Список отличников группы

ИТМО

```
class Student {
  ...

  public static void printAll() { ... }

  public static void printExcellentFromGroup(String g) {
    for (Student st : students) {
      if ((st.getGroup().equals(g) && (st.getAvgMark() > 4.75)) {
        System.out.println(st.getName());
      }
    }
  }
}
```

Java

Список молодых отличников

ИТМО

```
class Student {
  ...

  public static void printAll() { ... }

  public static void printExcellentFromGroup(String g) {...}

  public static void printExcellentAndYoung() {
    for (Student st : students) {
      if ((st.getAvgMark() > 4.75) && (st.getAge() < 20)) {
        System.out.println(st.getName());
      }
    }
  }
}
```

Java

Список молодых отличников

ИТМО

```
for (Student st : students) {
  if ((st.getAvgMark() > 4.75) && (st.getAge() < 20)) {
    System.out.println(st.getName());
  }
}
```

```
for (Student st : students) {
  if ((st.getAvgMark() > 4.75) && (st.getAge() < 20)) {
    System.out.println(st.getName());
  }
}
```

```
for (Student st : students) {
  if () {
    System.out.println(st.getName());
  }
}
```

Java

Список избранных :)

ИТМО

```
interface Checker {
  abstract public boolean test(Student st);
}

class Student {
  ...
  public static void printSelected(Checker ch) {
    for (Student st : students) {
      if (ch.test(st)) {
        System.out.println(st.getName());
      }
    }
  }
}
```

Java

Интерфейс и класс для отбора

ИТМО

```
interface Checker {
  abstract public boolean test(Student st);
}
class Student {
  ...
  public static void printSelected(Checker ch) {...}
}

class ExcellentAndYoungChecker implements Checker {
  public boolean test(Student st) {
    return (st.getAge() < 20) && (st.getAvgMark() > 4.75));
  }
}
Student.printSelected(new ExcellentAndYoungChecker());
```

Java

Анонимный класс

ИТМО

```
interface Checker {
  abstract public boolean test(Student st);
}
class Student {
  ...
  public static void printSelected(Checker ch) {...}
}

Student.print(new Checker() {
  public boolean test(Student st) {
    return (st.getAge() < 20) && (st.getAvgMark() > 4.75));
  }});
```

Java

Функциональный интерфейс

ИТМО

```
@FunctionalInterface interface Checker {
  abstract public boolean test(Student st);
}
class Student {
  ...
  public static void print(Checker ch) {...}
}

Student.print(new Checker() {
  public boolean test(Student st) {
    return (st.getAge() < 20) && (st.getAvgMark() > 4.75));
  });
```

Java

Лямбда-выражение

ИТМО

```
@FunctionalInterface interface Checker {
  abstract public boolean test(Student st);
}
class Student {
  ...
  public static void print(Checker ch) {...}
}

Student.print(new Checker() {
  public boolean test(Student st) {
    return (st.getAge() < 20) && (st.getAvgMark() > 4.75));
  });
```

```
(Student st) ->
    (st.getAge() < 20) && (st.getAvgMark() > 4.75))
```

Java

Лямбда-выражение

ИТМО

```
@FunctionalInterface interface Checker {
  abstract public boolean test(Student st);
}
class Student {
  ...
  public static void print(Checker ch) {...}
}

Student.print((Student st) ->
              (st.getAge() < 20) && (st.getAvgMark() > 4.75)) );
```

Java

Лямбда-выражение

ИТМО

```
@FunctionalInterface interface Checker {
  abstract public boolean test(Student st);
}
class Student {
  ...
  public static void print(Checker ch) {...}
}

Student.print(st ->
              (st.getAge() < 20) && (st.getAvgMark() > 4.75)) );

Student.print(st -> st.getGroup() == 3145);
```

Java

java.util.function.*

ИТМО

```
interface java.util.function.Predicate<T> {
  abstract public boolean test(T t);
}
class Student {
  ...
  public static void print(Predicate<Student> ch) {...}
}

Student.print(
  st -> (st.getAge() < 20) && (st.getAvgMark() > 4.75))
);

Student.print(st -> st.getGroup() == 3145);
```

Java

Predicate - условие

ИТМО

```
import java.util.function.*;

class Student {
  ...
  public static void print(Predicate<Student> ch) {
    for (Student st : students) {
      if (ch.test(st)) {
        System.out.println(st.getName());
      }
    }
  }
}

Student.print(st -> st.getGroup().equals("3145"));
```

Java

+ Consumer - принять результат

ИТМО

```
import java.util.function.*;

class Student {
  ...
  public static void handle(Predicate<Student> p,
                            Consumer<Student> c) {
    for (Student st : students) {
      if (p.test(st)) { c.accept(st); }
    }
  }
}

Student.handle(st -> st.getGroup().equals("3145"),
               st -> System.out.println(st));
```

Java

+ Iterable - универсальный обработчик

ИТМО

```
import java.util.function.*;

class Student {
  ...
  public static <X> void handle(Iterable<X> i,
                                Predicate<X> p,
                                Consumer<X> c) {
    for (X e : i) {
      if (p.test(e)) { c.accept(e); }
    }
  }
}

Student.handle(Student.students,
               s -> s.getGroup().equals("3145"),
               s -> System.out.println(s));
```

Java

Ссылка на метод (Method Reference)

ИТМО

```
import java.util.function.*;

class Student {
  ...
  public static <X> void handle(Iterable<X> i,
                                Predicate<X> p,
                                Consumer<X> c);
  public boolean checkGroup(String g) {
    return this.getGroup().equals(g);
  }
}

Student.handle(Student.students,
               s -> s.checkGroup("3145"),
               s -> System.out.println(s));
```

Java

Ссылка на метод (Method Reference)

ИТМО

```
import java.util.function.*;

class Student {
  ...
  public static <X> void handle(Iterable<X> i,
                                Predicate<X> p,
                                Consumer<X> c);
  public boolean checkGroup(String g) {
    return this.getGroup().equals(g);
  }
}

Student.handle(Student.students,
               s -> s.checkGroup("3145"),
               System.out::println);
```

Java

Варианты ссылок на метод

ИТМО

- Аналог лямбда-выражения с вызовом метода
- Синтаксис: **`TypeName::methodName`**
  - Type::staticMethod
  - object::instanceMethod
  - Class::instanceMethod
  - Type.super::instanceMethod
  - Class::new
  - type[]::new
  - x -> Type.staticMethod(x)
  - x -> object.instanceMethod(x)
  - (Class x) -> x.instanceMethod()
  - x -> Type.super.instanceMerhod(x)
  - x -> new Class(x)
  - x -> new type[x]

Java

@FunctionalInterface и λ

ИТМО

- λ-выражение — блок кода для объявления анонимной функции.
- λ-выражение имеет целевой тип функционального интерфейса
- Функциональный интерфейс - только один абстрактный метод
  - не считая default и методов Object

Java

@FunctionalInterface и λ

ИТМО

- λ-выражение можно присвоить переменной
  - Comparator<Integer> comp = (x, y) -> y - x;
- λ-выражение можно передать методу
  - **public static <T> void sort(T[] a, Comparator<? super T> c)**
  - Arrays.sort(array, (s1, s2) -> s1.length() - s2.length);
  - Arrays.sort(array2, comp);
- λ-выражение можно вернуть из метода
  - **public static <T> Comparator<T> reverseOrder()**
  - Arrays.sort(array3, Collections.reverseOrder());

Java

Синтаксис λ-выражений

ИТМО

```
параметр -> выражение

(параметры) -> { инструкции; }

(int x, int y) -> x + y

(x, y) -> x * y

() -> 42

(String s) -> System.out.println(s)

x -> x / 2

c -> { int s = c.size(); c.clear(); return s; }
```

Java

Захват переменных

- Область видимости λ-выражения = область видимости окружающего блока
- В λ-выражении можно использовать только эффективно финальные переменные из окружающего его блока
- λ-выражение захватывает значения переменных из окружающего блока.
- λ-выражение + значения захваченных переменных = замыкание (closure)

```
public static void repeatMessage(int count, String message) {
  Runnable r = () -> {
    for (int i = 0; i < count; i++) { println(message); }
  }
new Thread(r).start();
}
```